

Univerzitet u Beogradu
Matematički fakultet

Package Analyzer

Analizator zavisnosti softverskih paketa

Seminarski rad iz predmeta
Dizajn i analiza algoritama

Student: Valentina Dragić

Indeks: 078/2022

Jezik: C++20

Algoritmi: BFS, Tarjanov SCC, Kahnov topsort

Beograd, 2026.

Contents

1	Uvod	2
1.1	Motivacija	2
2	Opis problema i ulazni format	2
2.1	Registar zavisnosti	2
2.2	Lista potrebnih paketa	3
2.3	Matematički model	3
3	Arhitektura projekta	3
3.1	Struktura čvora	3
3.2	Graf kao jedini model	4
4	Algoritmi	4
4.1	Tarjanov algoritam za jako povezane komponente	4
4.1.1	Teorijska osnova	4
4.1.2	Opis algoritma	4
4.1.3	Implementacija	5
4.1.4	Zašto Tarjan, a ne Kahn za detekciju ciklusa	6
4.2	Kahnov algoritam za topološko sortiranje	6
4.2.1	Teorijska osnova	6
4.2.2	Adaptacija za obrnutu semantiku ivica	6
4.3	BFS za tranzitivne zavisnosti	7
4.3.1	Teorijska osnova	7
5	Interaktivni interfejs	8
6	Poređenje sa pravim alatima	9
7	Zaključak	9

1. Uvod

Svaki savremeni softverski ekosistem — Python, Node.js, Rust, Java — oslanja se na *menadžere paketa* koji automatski razrešavaju zavisnosti između biblioteka. Kada korisnik zatraži instalaciju paketa `flask`, sistem mora da shvati da je potrebno instalirati i `werkzeug`, `jinja2`, `click`, i sve njihove tranzitivne zavisnosti, i to u ispravnom redosledu.

Ovaj problem se prirodno modelira kao problem na usmerenim grafovima:

- **Čvor** predstavlja jedan softverski paket
- **Usmerena ivica** $A \rightarrow B$ znači da paket A zavisi od paketa B , odnosno da B mora biti instaliran pre A

Projekat implementira konzolni analizator koji gradi takav graf iz tekstualnog registra zavisnosti i primenjuje klasične algoritme teorije grafova za analizu: Tarjanov algoritam za pronalaženje jako povezanih komponenti, Kahnov algoritam za topološko sortiranje i BFS za analizu dostižnosti.

1.1. Motivacija

Isti grafovski problemi koji se rešavaju u ovom projektu pojavljuju se u industrijskim alatima:

Problem	Ovaj projekat	Pravi alati
Kružne zavisnosti	Tarjanov SCC	pip, npm, apt
Redosled instalacije	Kahnov topsort	pip, npm, Maven
Tranzitivne zavisnosti	BFS	pip show, npm list

Table 1: Poređenje sa industrijskim alatima

2. Opis problema i ulazni format

2.1. Registar zavisnosti

Ulazni fajl `packages.txt` sadrži registar svih poznatih paketa u formatu:

```

1 # Komentar
2 flask: werkzeug, jinja2, click, itsdangerous
3 jinja2: markupsafe
4 werkzeug: markupsafe
5 requests: urllib3, certifi, charset-normalizer, idna
6 click:

```

Listing 1: Format registra zavisnosti

Svaki red definiše jedan paket i listu njegovih direktnih zavisnosti odvojenih zarezima. Paketi bez zavisnosti mogu imati praznu desnu stranu ili je izostaviti. Redovi koji počinju sa `#` i prazni redovi se ignorišu.

2.2. Lista potrebnih paketa

Fajl `needed.txt` sadrži listu paketa koje korisnik želi da instalira:

```
1 # REST API projekat
2 fastapi
3 sqlalchemy
4 pyjwt
5 celery
```

Listing 2: Format liste potrebnih paketa

Program automatski pronalazi sve tranzitivne zavisnosti navedenih paketa.

2.3. Matematički model

Neka je P skup svih paketa i $E \subseteq P \times P$ skup zavisnosti. Graf zavisnosti $G = (P, E)$ je usmeren graf gde ivica $(u, v) \in E$ znači da paket u zavisi od paketa v .

Cilj je:

1. Detektovati sve jako povezane komponente veličine ≥ 2 (kružne zavisnosti)
2. Pronaći topološko uređenje skupa traženih paketa (redosled instalacije)
3. Za dati paket p pronaći skup $\text{dep}^*(p) = \{v \in P \mid v \text{ je dostižan iz } p\}$ (sve tranzitivne zavisnosti)

3. Arhitektura projekta

Projekat je organizovan po principu jedne odgovornosti (*Single Responsibility Principle*) — svaki modul ima tačno jednu ulogu.

Modul	Odgovornost
<code>paket.hpp</code>	Struktura čvora grafa: naziv i id
<code>parser.hpp/cpp</code>	Čitanje fajlova, vraća edge list
<code>graph.hpp/cpp</code>	Graf kao jedini model podataka
<code>algoritmi.hpp/cpp</code>	Tarjan, Kahn, BFS — slobodne funkcije
<code>komande.hpp/cpp</code>	Logika svake korisničke komande
<code>petlja.hpp/cpp</code>	Interaktivna petlja
<code>main.cpp</code>	Pokretanje (25 linija)

Table 2: Struktura modula

3.1. Struktura čvora

Čvor grafa je minimalna struktura — samo naziv i identifikator:

```
1 struct Paket {
2     std::string naziv;
3     int id;
4 }
```

```

5   Paket() : naziv(""), id(-1) {}
6   Paket(const std::string& naziv, int id)
7       : naziv(naziv), id(id) {}
8 };

```

Listing 3: paket.hpp

3.2. Graf kao jedini model

Ključna dizajnerska odluka je da **Graf** bude jedini model podataka. Parser vraća sirove stringove (edge list), graf ih prima i dodeljuje identifikatore, a sav ostali kod komunicira isključivo kroz te identifikatore. Ovo eliminiše sinhronizaciju između više kopija istih podataka.

```

1  class Graf {
2  public:
3      void popuni(const std::vector<Ivica>& ivice,
4                  const std::vector<std::string>& svaPaketi);
5
6      int brojPaketa() const;
7      const Paket& paket(int id) const;
8      const std::vector<int>& susedi(int id) const;
9      int nadjiId(const std::string& naziv)
10         const;
11     bool postojiPaket(const std::string&
12         naziv) const;
13 };

```

Listing 4: Javni interfejs klase Graf

4. Algoritmi

4.1. Tarjanov algoritam za jako povezane komponente

4.1.1. Teorijska osnova

Jako povezana komponenta (SCC) usmerenog grafa $G = (V, E)$ je maksimalan skup čvorova $C \subseteq V$ takav da za svaka dva čvora $u, v \in C$ postoji put od u do v i put od v do u .

U kontekstu zavisnosti paketa, SCC veličine ≥ 2 predstavlja *kružnu zavisnost* — paketi u toj komponenti čekaju jedan na drugog i nije ih moguće instalirati.

4.1.2. Opis algoritma

Tarjanov algoritam pronalazi sve SCC-ove u jednom DFS prolasku složenosti $O(V + E)$. Svaki čvor dobija dva broja:

- $\text{redDolaska}[v]$ — redni broj prvog poseta čvora v
- $\text{lowlink}[v]$ — najmanji redDolaska dostižan iz podstabla čvora v (uključujući v sam)

Čvor v je koren svoje SCC ako i samo ako važi:

$$\text{lowlink}[v] = \text{redDolaska}[v]$$

Kada se otkrije koren, svi čvorovi iznad njega na steku čine jednu SCC.

4.1.3. Implementacija

Ključna implementaciona odluka je da se brojač prolaska prenosi *referencom* kroz rekurziju, čime se eliminiše globalno stanje:

```

1 static void tarjanDFS(int cvor,
2                       const Graf& graf,
3                       int& brojac,           // referenca, ne
                                                globalna promenljiva
4                       std::vector<int>& redDolaska,
5                       std::vector<int>& lowlink,
6                       std::vector<bool>& naStek,
7                       std::stack<int>& stek,
8                       std::vector<std::vector<int>>&
                           komponente) {
9
10    redDolaska[cvor] = brojac;
11    lowlink[cvor]    = brojac;
12    brojac++;
13
14    naStek[cvor] = true;
15    stek.push(cvor);
16
17    for (int sused : graf.susedi(cvor)) {
18        if (redDolaska[sused] == -1) {
19            // neposecen cvor -- idemo dublje
20            tarjanDFS(sused, graf, brojac, redDolaska,
21                    lowlink, naStek, stek, komponente);
22            lowlink[cvor] = std::min(lowlink[cvor],
23                                    lowlink[sused]);
24        } else if (naStek[sused]) {
25            // povratna grana -- cvor je u istoj komponenti
26            lowlink[cvor] = std::min(lowlink[cvor],
27                                    redDolaska[sused]);
28        }
29    }
30
31    // v je koren SCC ako je lowlink[v] == redDolaska[v]
32    if (lowlink[cvor] == redDolaska[cvor]) {
33        std::vector<int> komponenta;
34        while (true) {
35            int vrh = stek.top(); stek.pop();
36            naStek[vrh] = false;
37            komponenta.push_back(vrh);
38            if (vrh == cvor) break;
39        }

```

```

38     komponente.push_back(komponenta);
39 }
40 }

```

Listing 5: Tarjanov DFS – eliminacija globalnog stanja

4.1.4. Zašto Tarjan, a ne Kahn za detekciju ciklusa

Kahnov algoritam detektuje *postojanje* ciklusa kao sporedni efekat topološkog sortiranja — ako na kraju nisu obrađeni svi čvorovi, postoji ciklus. Međutim, Kahn ne daje informaciju o tome *koji* paketi čine ciklus.

Tarjanov algoritam daje lokaciju ciklusa — vraća tačno koje pakete treba popraviti. U registru sa stotinama paketa, razlika između “ima ciklus” i “paketA ↔ paketB ↔ paketC čine ciklus” je ključna za dijagnostiku.

Iz istog razloga Tarjanov algoritam koriste i industrijski build sistemi poput Bazel-a za prikaz cirkularnih zavisnosti između build targeta.

4.2. Kahnov algoritam za topološko sortiranje

4.2.1. Teorijska osnova

Topološko uređenje usmerenog acikličnog grafa (DAG) $G = (V, E)$ je linearno uređenje čvorova $v_1, v_2, \dots, v_n$ takvo da za svaku ivicu $(v_i, v_j) \in E$ važi $i < j$.

U kontekstu instalacije paketa: ako paket u zavisi od paketa v , tada v mora doći pre u u redosledu instalacije.

4.2.2. Adaptacija za obrnutu semantiku ivica

Naš graf ima ivicu $u \rightarrow v$ koja znači “ u zavisi od v ”, dakle v mora biti instaliran *pre* u . Direktna primena Kahnovog algoritma dala bi obrnut redosled, pa je neophodno raditi na invertovanom grafu.

```

1  static std::vector<int> kahnSort(const Graf& graf,
2                                  const std::vector<int>&
3                                  podskup) {
4      std::vector<bool> ukljucen(graf.brojPaketa(), false);
5      for (int id : podskup) ukljucen[id] = true;
6
7      // preostalo[u] = broj zavisnosti koje cvor u jos nije
8      // "resio"
9      std::vector<int> preostalo(graf.brojPaketa(), 0);
10     std::vector<std::vector<int>> invertovan(graf.brojPaketa());
11
12     for (int u = 0; u < graf.brojPaketa(); u++) {
13         if (!ukljucen[u]) continue;
14         for (int v : graf.susedi(u)) {
15             if (!ukljucen[v]) continue;
16             preostalo[u]++;
17             invertovan[v].push_back(u);
18         }
19     }
20 }

```

```

17     }
18
19     // paketi bez zavisnosti mogu biti instalirani odmah
20     std::queue<int> red;
21     for (int i = 0; i < graf.brojPaketa(); i++)
22         if (ukljucen[i] && preostalo[i] == 0) red.push(i);
23
24     std::vector<int> redosled;
25     while (!red.empty()) {
26         int cvor = red.front(); red.pop();
27         redosled.push_back(cvor);
28         for (int zavisnik : invertovan[cvor])
29             if (--preostalo[zavisnik] == 0) red.push(zavisnik);
30     }
31
32     return redosled; // prazan ako postoji ciklus
33 }

```

Listing 6: Kahnov algoritam na podskupu čvorova

Ako na kraju veličina rezultata nije jednaka veličini podskupa, postoji ciklus — Kahn to detektuje kao sporedni efekat, ali bez informacije o lokaciji ciklusa (zbog čega koristimo i Tarjana).

4.3. BFS za tranzitivne zavisnosti

4.3.1. Teorijska osnova

Za dati paket p , skup tranzitivnih zavisnosti je:

$$\text{dep}^*(p) = \{v \in V \mid \exists \text{ put od } p \text{ do } v \text{ u } G\} \setminus \{p\}$$

BFS iz čvora p po ivicama grafa zavisnosti daje tačno ovaj skup. Složenost je $O(V+E)$.

```

1  std::vector<int> tranzitivneZavisnosti(const Graf& graf, int
    startId) {
2      std::vector<bool> posecen(graf.brojPaketa(), false);
3      std::queue<int> red;
4
5      posecen[startId] = true;
6      red.push(startId);
7
8      std::vector<int> rezultat;
9
10     while (!red.empty()) {
11         int cvor = red.front(); red.pop();
12         for (int sused : graf.susedi(cvor)) {
13             if (!posecen[sused]) {
14                 posecen[sused] = true;
15                 red.push(sused);
16                 rezultat.push_back(sused);
17             }

```



```

18         }
19     }
20
21     return rezultat;
22 }

```

Listing 7: BFS za tranzitivne zavisnosti

BFS se koristi na dva mesta: u komandi `deps` za direktan prikaz zavisnosti, i interno u `skupiPodskup` za određivanje skupa paketa koji će biti analizirani pri `install` i `needed` komandama.

5. Interaktivni interfejs

Program nudi interaktivni konzolni interfejs — graf se učitava jednom, a korisnik postavlja upite bez ponovnog pokretanja:

Komanda	Opis	Algoritam
<code>install <paket></code>	Redosled instalacije	Kahn
<code>deps <paket></code>	Tranzitivne zavisnosti	BFS
<code>info <paket></code>	Direktne zavisnosti	Pristup listi susedstva
<code>ciklusi</code>	Kružne zavisnosti u registru	Tarjan SCC
<code>izolovani</code>	Paketi bez veza	Linearna provera
<code>needed <fajl></code>	Instalacija iz fajla	BFS + Kahn

Table 3: Dostupne komande

Primer sesije:

```

1 $ ./dep-analyzer packages.txt
2 Registar ucitan: 88 paketa, 101 zavisnosti.
3
4 KRUZNE ZAVISNOSTI
5 Nema kruzni zavisnosti. Graf je aciklican (DAG).
6
7 > install flask
8 Redosled instalacije za 'flask':
9 1. click
10 2. itsdangerous
11 3. markupsafe
12 4. jinja2
13 5. werkzeug
14 6. flask
15
16 > deps requests
17 Tranzitivne zavisnosti za 'requests' (4):
18 - urllib3
19 - certifi
20 - charset-normalizer

```

```
21 - idna
22
23 > install nepostojeci
24 Greska: paket 'nepostojeci' nije u registru.
```

Listing 8: Primer interaktivne sesije

6. Poređenje sa pravim alatima

Projekat implementira algoritmsku osnovu svakog paket menadžera. Razlika između ovog projekta i alata poput `pip`-a ili `npm`-a nije u algoritmima — isti Tarjan, Kahn i BFS se koriste — već u perifernim delovima koji nisu tema kursa teorije grafova:

- **Mrežni pristup** — pravi alati preuzimaju pakete sa centralnog repozitorijuma (PyPI, npm registry, apt sources)
- **Verzionisanje** — pravi alati podržavaju semver ograničenja poput `werkzeug >= 2.0, < 3.0`
- **Razrešavanje konflikata** — moderni `pip` koristi SAT solver (backtracking) za pronalaženje kompatibilnog skupa verzija; ovo je NP-kompletni problem u opštem slučaju
- **Izolacija okruženja** — `virtualenv`, `venv`, `node_modules` po projektu

Ove razlike su namerno izostavne — fokus projekta je na grafovskim algoritmima, ne na implementaciji kompletnog paket menadžera.

7. Zaključak

Projekat demonstrira da se problem upravljanja zavisnostima softverskih paketa može svesti na klasične probleme teorije grafova: detekciju jako povezanih komponenti, topološko sortiranje i analizu dostižnosti.

Implementirani algoritmi — Tarjanov SCC, Kahnov topsort i BFS — imaju složenost $O(V + E)$ i direktno odgovaraju algoritmima koji se koriste u industrijskim paket menadžerima. Arhitektura projekta je organizovana po principu jedne odgovornosti, bez globalnog stanja i sa jasnom podelom između strukture podataka, algoritama i korisničkog interfejsa.

References

- [1] Filip Marić, *Klasični i Aproksimativni Algoritmi*, Matematički fakultet, Univerzitet u Beogradu. Dostupno na: <https://poincare.matf.bg.ac.rs/~filip.maric/kiaa/kiaa.pdf>
- [2] Robert Tarjan, *Depth-First Search and Linear Graph Algorithms*, SIAM Journal on Computing, 1(2):146–160, 1972.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*, 4th edition, MIT Press, 2022.
- [4] Python Packaging Authority, *pip — The Python Package Installer*, <https://pip.pypa.io>